

Aletia Index

Clinical assurance for digital health tools.

The Aletia Index is a specialised Regulatory Information Management (RIM) and Clinical Trust platform. It serves as the reference standard for Artificial Intelligence and Machine Learning-enabled Medical Devices (AI/ML-MD) — bridging the gap between regulatory clearance and real-world clinical utility across multiple jurisdictions.

- **Live site:** <https://www.aletia-index.com>
 - **Staging:** <https://aletia-index.vercel.app>
 - **Repository:** <https://github.com/AnnMaidment/Aletia-Index>
-

What This Is

A searchable, publicly accessible index of AI/ML medical devices. Each device has its own SEO-indexed page showing:

- Regulatory clearance status (FDA, MHRA, CE Mark, SAHPRA)
 - Cross-jurisdiction availability and compliance pathway
 - Accountability tier (Level 1-5, from Clinical Decision Support to Autonomous Action)
 - Aletia Verified badge (awarded after human-in-the-loop clinical audit)
 - Green / Amber / Red health status
 - Data freshness timestamps (automated sync vs last clinical review)
 - Technical profile (FHIR, EHR compatibility, data hosting, POPIA compliance)
 - Pre-clearance pipeline stage indicator for devices not yet cleared
 - Data source provenance (Registry Sync / Aletia Research / Manufacturer Submitted)
-

Tech Stack

Layer	Technology	Purpose
Frontend	Next.js 16 + TypeScript	Web application framework
Styling	globals.css + CSS variables	Consolidated shared stylesheet
Database	PostgreSQL via Supabase	Device, manufacturer, audit data
Auth	Supabase Auth (planned)	Manufacturer & auditor logs

Layer	Technology	Purpose
Payments	Peach Payments (planned)	Audit fees & subscriptions (note: Stripe not available in ZA)
Hosting	Vercel	Automatic deployment from GitHub
Domain	Hostinger → Vercel DNS	aletia-index.com

Project Structure

```
aletia-index/
├─ app/
│   └─ page.tsx                # Main public index page – server component,
URL-based filtering
│   └─ layout.tsx              # Root layout (fonts, metadata, OG tags)
│   └─ globals.css              # Global stylesheet – all shared styles live
here
│   └─ components/
│       └─ FiltersBar.tsx      # Search, specialty, source, status filters
(debounced)
│       └─ DeviceGrid.tsx      # Table rows, pagination, modal quick-view
│       └─ QuickFilters.tsx    # Sidebar specialty pills
│       └─ MobileMenu.tsx      # Hamburger toggle
│   └─ device/
│       └─ [id]/
│           └─ page.tsx        # Individual device pages (SSR, SEO-optimised)
│   └─ api/
│       └─ fda-sync/
│           └─ route.ts        # Multi-jurisdiction sync API endpoint
│   └─ methodology/
│   └─ insights/
│   └─ clinicians/
│   └─ request-review/
├─ components/
│   └─ NavFooter.tsx          # Shared nav and footer component
├─ lib/
│   └─ supabase.ts            # Supabase client initialisation
│   └─ fda.ts                 # openFDA API service layer (57 AI/ML product
codes)
│   └─ fdaSync.ts             # FDA → Supabase data translator
│   └─ mhra.ts                # MHRA PARD API service layer
│   └─ mhraSync.ts            # MHRA → Supabase data translator
│   └─ eudamed.ts             # EUDAMED API (on-demand lookup only)
│   └─ eudamedSync.ts         # EUDAMED sync helpers
│   └─ types.ts               # Shared TypeScript types (Device, etc.)
```

```
├─ docs/
│   └─ MHRA_MANUFACTURER_ENRICHMENT.md # Manual enrichment tracker
├─ public/
│   └─ assets/ # Logo, favicon, OG image (lowercase filenames)
├─ .env.local # Environment variables (NOT in git)
└─ README.md
```

Styling Architecture

All shared CSS lives in `app/globals.css`. Page-specific styles use small inline blocks only where genuinely unique to that page.

CSS variables (used consistently across all files):

```
css

--primary: #1f6feb /* main blue */
--bg:      #f5f7fb /* page background */
--text:    #0f172a /* dark text */
--muted:   #64748b /* grey text */
--line:    #e6ebf3 /* border colour */
```

SEO Architecture

Each device has a permanent URL at `/device/[device_id]` rendered via SSR. This means:

- Every device is individually indexable by Google
- Pages always show live data (no stale static builds)
- New devices are immediately accessible — no redeployment needed

Each device page includes:

- Unique `<title>` and `<meta description>` via `generateMetadata`
- Open Graph tags for social sharing
- JSON-LD structured data (`MedicalDevice` schema)

Target keyword clusters:

- `[device name] FDA clearance`
- `[device name] approved South Africa`
- `is [device name] approved in [country]`
- `[specialty] AI medical device MHRA registered`

Database Schema

Seven tables in PostgreSQL (hosted on Supabase):

MANUFACTURERS

Column	Type	Notes
id	UUID, PK	
name	text	NOT NULL
hq_location	text	
status	text	Verified / Unverified — default 'Unverified'
payment	text	Payment provider customer ID — future billing
tier	text	free / claimed / premium / featured — default 'free'
claimed_at	timestamptz	When manufacturer claimed their listing
claimed_by_email	text	Email used to claim
claim_token	uuid	One-click claim link token
website	text	
funding_stage	text	Seed / Series A / Series B / Grant — for pipeline listings
total_raised_usd	numeric	Approximate, from public sources
contact_name	text	
contact_email	text	
contact_visible	boolean	Whether to show contact button publicly
alert_email	text	Registration expiry alert recipient
alert_active	boolean	Whether expiry alerts are enabled
created_at	timestamptz	default now()

SPECIALTY_TAXONOMY

Column	Type	Notes
specialty_name	PK	
parent_cat	text	e.g. Psychiatry → Mental Health

DEVICE_MASTER

Central table. One row per device.

Column	Type	Notes
device_id	PK	FDA K number, MHRA-{ID}, etc.
manufacturer_link	FK → manufacturers	nullable
manufacturer_name	text	Plain text name — fallback when FK not set, backfilled from manufacturers.name
search_text	text GENERATED	Lowercase concat of device_id + intended_use + manufacturer_name — used for full-text search
specialty_link	FK → specialty_taxonomy	nullable
mode	text	Standalone / Integrated
dependency	text	EHR / Hardware / Cloud
autonomy	text	Solo / Assistant
intended_use	text	
ai_ml_type	text	Locked / Continuous Learning
accountability_tier	int	1-5
health_status	text	Green / Amber / Red — default 'Amber'
aletia_verified	boolean	default false
country_of_origin	text	'US', 'GB', 'EU', 'ZA'
basic_udi_ulid	text	EUDAMED UDI — for on-demand EU lookup
udi	text	Cross-jurisdiction UDI — future matching key
last_automated_sync	timestamptz	
last_clinical_review	timestamptz	
pipeline_stage	text	(proof_of_concept) / (pre_submission) / (submitted) / (under_review) / (cleared) — NULL for registry-synced cleared devices

Column	Type	Notes
data_source	text	<code>registry_sync</code> / <code>manufacturer_submitted</code> / <code>aletia_research</code> — drives disclaimer logic — default 'registry_sync'
excluded	boolean	NOT NULL default false — soft-exclude from all queries without deleting
excluded_reason	text	<code>not_ai_ml</code> / <code>duplicate</code> / <code>out_of_scope</code> / <code>test_record</code>
breakthrough_designation	boolean	default false — FDA Breakthrough Device Programme
breakthrough_designation_date	date	
created_at	timestampz	default now()

Note on `pipeline_stage`: Currently stored at device level only. A future migration may move this to `regional_registrations` to support per-jurisdiction pipeline tracking (e.g. cleared in US but under review in EU). This was a deliberate deferral — migrate when pipeline data volume justifies the complexity.

REGIONAL_REGISTRATIONS

One row per jurisdiction per device. A device cleared in UK and US has two rows.

Column	Type	Notes
reg_id	UUID, PK	
device_link	FK → device_master	
country	text	'US', 'GB', 'EU', 'ZA'
regulatory_body	text	FDA / MHRA / CE Mark / SAHPRA
clearance_type	text	510k / PMA / De Novo / UKCA / MDR / MDD / Acknowledgement Letter / Full Registration / pipeline
device_class	text	Class I / IIa / IIb / III
gmdn_code	text	GMDN code
gmdn_term	text	GMDN term name
last_updated	timestamptz	Last update from source registry
regulatory_expiry	date	 RED FLAG — alert when expiring within 90 days
local_distributor	text	
adverse_event_count	integer	From FDA MAUDE / MHRA adverse event data
adverse_event_source	text	
recall_active	boolean	default false
recall_detail	text	

Unique constraint: (device_link, country, regulatory_body)

CLINICAL_AUDITS

Column	Type	Notes
audit_id	UUID, PK	default gen_random_uuid()
device_link	FK → device_master	
evidence_review_date	timestamptz	 RED FLAG — alert when stale
confidence_score	float	1-10
evidence_log	text	PDF / links

Column	Type	Notes
peer_reviewed_validation	int	checklist score
demographic_accuracy	int	checklist score
sahpra_compliance	int	checklist score
cybersecurity_popia	int	checklist score
created_at	timestamptz	default now()

TECH_SPECS

Column	Type	Notes
device_link	PK/FK → device_master	
api_type	text	FHIR / REST
ehr_compat	text	Epic, TrakCare, etc.
data_hosting	text	Local / Cloud
fhir_compatible	boolean	
popia_compliant	boolean	

Search Architecture

The main index uses a **generated column** `search_text` on `device_master` for server-side full-text search:

```

sql
ALTER TABLE device_master
ADD COLUMN search_text text GENERATED ALWAYS AS (
  lower(
    coalesce(device_id, '') || ' ' ||
    coalesce(intended_use, '') || ' ' ||
    coalesce(manufacturer_name, '')
  )
) STORED;

CREATE INDEX idx_device_search_text ON device_master USING gin(to_tsvector('english'
```

The search query uses `.ilike('search_text', '%term%')`. Because `manufacturer_name` is backfilled from `manufacturers.name` for all devices with a `manufacturer_link`, this catches searches by manufacturer name even though the join table is not directly filterable via PostgREST `.or()`.

Backfill command (run after any bulk seed to populate `manufacturer_name` from join):

```
sql

UPDATE device_master dm
SET manufacturer_name = m.name
FROM manufacturers m
WHERE dm.manufacturer_link = m.id
AND dm.manufacturer_name IS NULL;
```

Page Architecture

The main index (`app/page.tsx`) is a **server component** that reads URL search params, runs parallel Supabase queries, and passes data to client islands:

- `FiltersBar` — search (350ms debounce), specialty/source selects, status pills. Uses `router.replace` to update URL without stacking history.
- `DeviceGrid` — table rows, Link-based pagination, modal quick-view.
- `QuickFilters` — sidebar specialty pills.
- `MobileMenu` — hamburger toggle only.

Filters are URL-driven: `/?search=cardio&specialty=Radiology&status=Green&page=2`

The sidebar stats (donut chart, counts) are fetched with `.range(0, 9999)` to bypass Supabase's default 1000-row cap. Without this the stats reflect only the first 1000 devices.

API Integrations

Jurisdiction Overview

Regulatory Body	Country	Clearance Types	Status
FDA	US	510k, PMA, De Novo	 Production
MHRA	GB	UKCA (Class I-III)	 Production

Regulatory Body	Country	Clearance Types	Status
CE Mark	EU	MDR, MDD	⚠️ On-demand lookup only
SAHPRA	ZA	Acknowledgement Letter, Full Registration	📅 Planned

FDA Integration

Source: openFDA API — <https://open.fda.gov/apis/device/>

Scope: Class II and Class III AI/ML devices only. Class I excluded.

Files:

- `lib/fda.ts` — API service layer
- `lib/fdaSync.ts` — Data translator

Product codes: `fda.ts` contains a curated list of **57 FDA product codes** associated with AI/ML medical devices, derived from the FDA's official AI-Enabled Medical Devices list (last updated March 2026). The original list had 6 codes — expanding to 57 increased the seeded device count from ~175 to 5,037. The list is grouped by clinical area and should be reviewed quarterly as the FDA publishes new authorisations.

Search strategy (three-tiered):

1. `searchAIMLByProductCodes()` — primary. Batches 57 codes into groups of 15 with 300ms delays. Most reliable — catches devices regardless of product name.
2. `searchAIMLDevices()` — supplementary. Keyword search on device name for AI/ML terms. Catches devices in unlisted product codes.
3. `searchAllAIMLDevices()` — combined. Runs both and deduplicates by K number. **Use this for full reseeds** via `PUT /api/fda-sync`.

Health status logic:

- ● Red — active recall present
- ● Amber — adverse event count exceeds 50 MDRs
- ● Green — no signals

Critical fix (March 2026): `fdaSync.ts` previously used `.update()` which silently did nothing when a device didn't exist in the database. Changed to `.upsert()` with `onConflict: 'device_id'`. New devices (e.g. K251293 CardioVision by Icardio.Ai) are now inserted on first

sync rather than silently dropped. `intended_use`, `manufacturer_name`, `data_source`, and `excluded` are all set on insert.

Why K251293 was missing: The device name "CardioVision" contains no AI/ML keywords, and its product code `QU0` (Heart Failure Status Indicator) was not in the original 6-code list. Both issues are now resolved.

MHRA PARD Integration

Source: MHRA Public Access Registration Database — <https://pard.mhra.gov.uk>

Scope: AI/ML SaMD devices across 35 curated GMDN terms. Class I included.

Files:

- `lib/mhra.ts` — API service layer (POST `/searchDevices`, POST `/searchManufacturers`)
- `lib/mhraSync.ts` — Data translator

Health status logic:

- Red — registration lapsed (status $\neq$ DREGIY)
- Amber — active but `LAST_UPDATED_DATE` is more than 2 years ago
- Green — active registration, updated within 2 years

Manufacturer names: PARD's device API returns `MAN_ORGANISATION_ID` only. All 44 previously unresolved manufacturer names have been manually enriched using `pard.mhra.gov.uk/manufacturer-details/\[ID\]` and updated via SQL. The enrichment process is documented in `docs/MHRA_MANUFACTURER_ENRICHMENT.md`.

EUDAMED — On-Demand Lookup Only

Source: European database on medical devices — <https://ec.europa.eu/tools/eudamed>

Why not bulk seeded: EUDAMED's API does not support reliable filtering for AI/ML devices. Bulk queries return up to 1.2 million devices with no AI/ML-specific filter.

Current use: On-demand lookup only. Given a `basic_udi_ulid`, returns registration details for a specific device.

Files: `lib/eudamed.ts`, `lib/eudamedSync.ts`

The Sync Endpoint

`app/api/fda-sync/route.ts` — secured with `SYNC_SECRET` header `x-sync-token`.

Method	URL	Action
GET	<code>/api/fda-sync</code>	Health check / endpoint listing
GET	<code>/api/fda-sync?bulk=true</code>	Re-sync all FDA devices already in regional_registrations
POST	<code>/api/fda-sync</code>	Sync or insert a single FDA device by K number
PUT	<code>/api/fda-sync</code>	Full reseed — runs <code>searchAllAIMLDevices()</code> (product codes + name search)
PATCH	<code>/api/fda-sync</code>	Backfill <code>manufacturer_name</code> for all existing K-number devices with null name

PowerShell examples (use on Windows — curl is aliased to Invoke-WebRequest):

```
powershell

# Full reseed - pulls all AI/ML devices from FDA using 57 product codes
Invoke-WebRequest -Uri "http://localhost:3000/api/fda-sync" -Method PUT `
  -Headers @{ "x-sync-token" = "your_secret_here" }

# Sync or insert a single device by K number
Invoke-WebRequest -Uri "http://localhost:3000/api/fda-sync" -Method POST `
  -Headers @{ "Content-Type" = "application/json"; "x-sync-token" = "your_secret_here" }
  -Body '{"device_id": "K251293", "k_number": "K251293"}'

# Backfill manufacturer names
Invoke-WebRequest -Uri "http://localhost:3000/api/fda-sync" -Method PATCH `
  -Headers @{ "x-sync-token" = "your_secret_here" }
```

⚠ Critical notes on FDA syncing:

- **OPENFDA_API_KEY must be set** in both `.env.local` and Vercel environment variables. Without a valid key the FDA API returns HTTP 403 and all individual device lookups silently return only `health_status` and `last_automated_sync` — FDA data (manufacturer name, intended use, device class) is never written. Get a free key at <https://open.fda.gov/apis/authentication/>
- The PUT reseed calls `searchAllAIMLDevices()` which batches 57 product codes. Responses exceed Next.js's 2MB fetch cache limit — this produces `Failed to set fetch cache` warnings in the console. These are warnings only, not errors — the data comes through correctly. The `cache: 'no-store'` option in `fdaFetch()` suppresses these warnings.

- The `fdaSync.ts` upsert sets `excluded: false` and `data_source: 'registry_sync'` on every insert. Manually excluded devices (`excluded: true`) will be re-included if their K number appears in a reseed. Set `excluded` back to true after reseeding if needed.
 - The POST single-device sync will **insert** devices not yet in the database (fixed March 2026). Previously it used `.update()` and silently did nothing for new devices.
-

Cross-Jurisdiction Device Matching

Different registries use different identifiers — FDA uses K numbers, MHRA uses numeric DEVICE_IDs. There is no direct shared key today.

The future bridge is UDI (Unique Device Identifier). The US, UK, EU, and most major markets are converging on UDI as a global standard. `device_master` has a `udi` column reserved for this purpose.

GMDN codes (present in `regional_registrations`) provide a weaker but available cross-jurisdiction link at the device *type* level today.

Getting Started (Local Development)

Prerequisites

- Node.js v20+ — <https://nodejs.org>
- Git — <https://git-scm.com>
- A Supabase account — <https://supabase.com>

1. Clone the repository

```
bash
git clone https://github.com/AnnMaidment/Aletia-Index.git
cd Aletia-Index
```

2. Install dependencies

```
bash
npm install
```

3. Set up environment variables

Create a `.env.local` file in the root:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_publishable_key_here
OPENFDA_API_KEY=your_fda_api_key_here    # Get free key at
https://open.fda.gov/apis/authentication/
SYNC_SECRET=your_sync_secret_here
```

4. Run the development server

```
bash

npm run dev
```

Open <http://localhost:3000>.

Deployment

The project auto-deploys to Vercel on every push to `main`.

Variable	Where to get it	Required
NEXT_PUBLIC_SUPABASE_URL	Supabase → Settings → API	Yes
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase → Settings → API Keys	Yes
OPENFDA_API_KEY	https://open.fda.gov/apis/authentication/	Yes — without this individual device syncs silently fail
SYNC_SECRET	Any long random string you choose	Yes (production)

Branching Strategy

Never push directly to `main`. Always work on a feature branch and merge via pull request.

```
main          ← production (auto-deploys to Vercel)
dev           ← staging (test here before merging to main)
feature/xxx   ← individual features (branch from dev or main)
```

Roadmap

V1 Complete

- Public searchable device index
- Supabase database with real schema
- Device detail modal
- Search and filter by specialty and status
- Deployed to Vercel with custom domain
- All navigation pages (Methodology, Insights, Clinicians, Request Review)

V2 Complete

-  openFDA API integration — AI/ML devices seeded
-  MHRA PARD integration — UK AI/ML devices seeded
-  Multi-jurisdiction registration support
-  MHRA sync files committed to repository (mhra.ts, mhraSync.ts)
-  Multi-jurisdiction route.ts (FDA + MHRA + EUDAMED)
-  MHRA manufacturer names fully enriched (44 org IDs resolved)
-  CSS consolidated into globals.css
-  Individual device pages (`/device/[id]`) with SSR and SEO metadata
-  JSON-LD structured data on device pages
-  Open Graph tags across all pages
-  FDA manufacturer names backfilled via PATCH endpoint
-  Search null guard — (`toLowerCase()`) crash on null fields resolved
-  Pagination on main index — 25 devices per page
-  Manufacturer name fallback chain in table (`manufacturers.name`) → `manufacturer_name` → device ID)
-  Name and intended use truncation in table (2 and 3 lines respectively)

V3 Substantially Complete (March 2026)

-  FDA product code list expanded from 6 → 57 codes (source: FDA official AI/ML device list)
-  5,037 devices seeded (up from ~175)
-  `fdaSync.ts` upsert fix — new devices now insert correctly (was silently dropping new K numbers)
-  `excluded` and `excluded_reason` fields — soft exclusion without deleting records

- ☒ `pipeline_stage` and `data_source` fields — pre-clearance device support
- ☒ `breakthrough_designation` field — FDA Breakthrough Device flag
- ☒ Pipeline stepper on device pages — replaces health status for pre-clearance devices
- ☒ Pre-clearance disclaimer banner — persistent, non-dismissible, patient safety requirement
- ☒ Data source provenance tags (`Aletia Research`, `Manufacturer Submitted`)
- ☒ Claim this listing notice on device pages — data completeness framing
- ☒ Manufacturer schema extended — tier, claim fields, contact, alert, funding fields
- ☒ Regional registrations extended — `adverse_event_count`, `recall_active`, `regulatory_expiry`
- ☒ Server-side filtering and pagination — URL-driven, no client-side data loading
- ☒ `search_text` generated column — enables server-side full-text search
- ☒ Donut chart colours corrected to match health status (green/amber/red)
- ☒ `OPENFDA_API_KEY` required and documented — 403 errors were silently corrupting syncs
- ☐ Dynamic sitemap.xml for Google indexing
- ☐ Claim your listing flow (manufacturer portal entry point)
- ☐ SAHPRA integration
- ☐ About / Privacy / Terms pages
- ☐ Auth (Supabase Auth for manufacturer and auditor roles)
- ☐ Vercel Cron Job for nightly automated sync

V4 Future

- ☐ 10-Point Clinical Assurance Checklist audit workflow
- ☐ Peach Payments integration (payment provider for ZA — Stripe not available)
- ☐ Cross-jurisdiction UDI matching
- ☐ GDPR / HIPAA / UK data protection fields in schema
- ☐ NMPA (China) integration
- ☐ API licensing and data watermarking
- ☐ Audit trail / change log for `device_master` updates
- ☐ Role-based access control — `admin` / `editor` / `auditor` roles via Supabase RLS (required before hiring editorial or clinical staff)
- ☐ AI/ML relevance scoring — calculated field to distinguish devices where AI is integral vs incidental (e.g. dedicated diagnostic AI vs MRI scanner with AI image enhancement feature)

-  `pccp_flag` — Predetermined Change Control Plan indicator (publicly available in 2024+ 510(k) summaries)
 -  `validation_demographics` — training/validation population disclosure field
-

Pre-Clearance Pipeline

The Pre-Clearance Pipeline tracker extends Aletia beyond cleared devices to include AI/ML medical device startups that are actively pursuing regulatory approval. This serves three audiences: clinicians tracking devices they have been pitched before clearance arrives, startups wanting visibility and access to competitive intelligence, and procurement officers monitoring the incoming device landscape.

What a Pre-Clearance Listing Is

A pre-clearance listing is a device record in `device_master` where `pipeline_stage` is set to one of the following values rather than NULL:

proof_of_concept → pre_submission → submitted → under_review → cleared

These records are populated either from public sources (Breakthrough Device Designations, funding databases, accelerator cohort lists) or from data submitted by the manufacturer directly after claiming their listing.

`data_source` controls how the listing is presented to users:

Value	Meaning	Display treatment
<code>registry_sync</code>	Pulled automatically from FDA / MHRA / EUDAMED	Standard cleared listing
<code>manufacturer_submitted</code>	Manufacturer has claimed and populated their own record	"Manufacturer Submitted" badge
<code>aletia_research</code>	Populated by Aletia from public sources — not yet claimed	"Aletia Research — Unverified" banner

Pipeline Stage Indicator

Pre-clearance device pages display a horizontal progress stepper rather than the standard Green / Amber / Red health status. The current stage is highlighted in blue, past stages in green, future stages greyed out. This replaces the health status block entirely and is visually distinct.

Health Status Logic for Pre-Clearance Devices

Pre-clearance devices do not receive a Green / Amber / Red health status. Instead:

- **No health status assigned** until `pipeline_stage` is NULL and a registry sync confirms the clearance
- **Disclaimer banner** displayed persistently at the top of every pre-clearance listing — cannot be dismissed
- **Aletia Verified badge** is never awarded to a pre-clearance device

Disclaimer Architecture

The visual distinction between cleared and pre-clearance listings is a patient safety requirement, not a design preference:

- Pre-clearance pages show a persistent amber banner before any other content
- The `Pipeline` filter pill in the index table is visually distinct from status filters
- Risk column shows `—` for pipeline devices — no health status is assigned

Competitive Landscape Report

A paid PDF product generated from pre-clearance and cleared device data, filtered by specialty. Price: £800–1,500 as a one-off purchase. Production is manual for the first ten reports.

Key Concepts

Accountability Tier — A 1–5 scale classifying how autonomous the AI device is:

- Tier 1: Clinical Decision Support
- Tier 2: Diagnostic Aid
- Tier 3: Diagnostic Decision
- Tier 4: Autonomous Screening
- Tier 5: Autonomous Action

Dual-Timestamp Logic — Every device has two timestamps:

- `last_automated_sync` — when the API last updated the record
- `last_clinical_review` — when a human auditor last reviewed it

The gap between these two timestamps is displayed on device pages as a data freshness signal (e.g. "14mo gap between last registry sync and last clinical review").

Aletia Verified — A badge awarded only after a human auditor completes the 10-Point Clinical Assurance Checklist. It cannot be self-assigned by manufacturers.

Health Status — Automatically derived from registry signals but overridable by human auditors:

- FDA: Red = active recall. Amber = >50 MDRs. Green = no signals.

- **MHRA:** Red = lapsed registration. Amber = not updated in 2+ years. Green = active and current.

Excluded — Devices with `excluded = true` are hidden from all public queries but retained in the database for sync integrity. Use `excluded_reason` to document why. Common reason: `not_ai_ml` — device shares a product code with AI/ML software but is primarily hardware.

GMDN — Global Medical Device Nomenclature. A standardised international coding system for device types. Used by MHRA PARD as the primary search key.

Access & Credentials

To get access to the project you will need:

- **GitHub** — Ask @AnnMaidment to add you as a collaborator
 - **Supabase** — Ask for an invite to the `aletia-index` project
 - **Vercel** — Ask for an invite to the `annmaidments-projects` team
 - `.env.local` file — Shared privately (never committed to git)
-

Contact

Project Owner: Annemarie Maidment **Email:** annemarie.maidment@gmail.com

Aletia Index — Independent clinical assurance for digital health tools.